



프로젝트 확장

- 특정 감독(Director)별 평균 평점 분석
- 특정 배우(Star) 별 평균 평점 비교
- 특정 조건(예: 평점 8.0 이상 & 2000년 이후 제작) 충족하는 영화만 필터링

확장 prepare:

- 기존 Step1 데이터를 기반으로 **프로젝트 확장 분석용 컬럼** 포함
- **Director**, **Star** 컬럼을 추가로 선택하고, 기존 컬럼(**Title**, **Genre**, **Rate**)과 함께 NumPy 배열로 변환

처리 순서

1. CSV 파일 읽기 (**IMDB Top 1000.csv**)
2. 큰 따옴표 안/밖 쉼표 구분 파싱 (**parse_csv_line**)
3. 필요한 컬럼 선택:
 - 기존: **Title**, **Genre**, **Rate**
 - 확장: **Director**, **Star** 추가 선택
4. Title 전처리
 - 앞 숫자 제거
 - 괄호 안 연도 추출 (**Released_Year**)
5. 결측값 제거
6. 데이터 타입 변환
 - Rate → float
 - Released_Year → int
7. NumPy 배열 변환 (**dtype=object**)
8. 확장용 **.npz** 저장 (**data_array_extension.npz**)

코드

```
import numpy as np

# csv 파일 경로
csv_path = "../01_data/IMDB top 1000.csv"

# csv 파싱 함수 정의
# 큰 따옴표 안에 있는 쉼표와 바깥에 있는 쉼표 구분
def parse_csv_line(line):
    row = []      # 한 줄을 분리한 문자열 담을 리스트
    current = ""   # 현재 문자열 누적
    inside_quotes = False # 큰 따옴표 안에 있는지 여부 체크
    for char in line:
        if char == '"' and not inside_quotes: # 큰 따옴표 시작
            inside_quotes = True
        elif char == '"' and inside_quotes: # 큰 따옴표 종료
            inside_quotes = False
        elif char == ',' and not inside_quotes: # 따옴표 밖에 쉼표
            row.append(current) # 여태 누적된 문자열 리스트 안에 담기
            current = ""      # 누적 초기화
        else:
            current += char    # 문자 누적
    row.append(current)      # 마지막 문자열 리스트 안에 담기
    return row              # for문 밖에서 반환

# csv 파일 읽기
with open(csv_path,"r", encoding="utf-8") as f:
    lines = f.read().splitlines() # 줄 단위 리스트 생성

# 헤더와 데이터 분리 후
header = parse_csv_line(lines[0]) # 파싱 헤더 추출
data_lines = lines[1:]           # 데이터 줄만 따로 저장
```

```

# -----
# 기존 Step1에서 선택된 컬럼 + 확장용 컬럼 추가
# -----
title_idx = header.index("Title") # 1
genre_idx = header.index("Genre") # 4
rate_idx = header.index("Rate") # 5
cast_idx = header.index("Cast") # 8 → 확장 Step1 : 감독/배우 정보 파싱용

# 데이터 전처리 리스트 준비
processed_data = [] # 최종 NumPy 배열로 변환할 데이터 담을 리스트

# 데이터 줄 반복 처리
for line in data_lines:
    row = parse_csv_line(line) # 파싱하여 컬럼 분리

    # row 길이가 필요한 컬럼 중 가장 큰 인덱스보다 작으면 해당 컬럼이 없다는 의미
    if len(row) <= max(title_idx, genre_idx, rate_idx):
        continue # 없다면, 건너 뛴다. 다음 줄을 처리하러 for문 처음으로 돌아감.

    # 컬럼 값 가져오기 및 공백 제거
    Num_title = row[title_idx].strip()
    genre = row[genre_idx].strip()
    rate = row[rate_idx].strip()

    # 확장 step1: Cast 컬럼에서 Direction/Stars 추출
    cast_info = row[cast_idx].strip()
    director = []
    stars = []
    if "Director:" in cast_info and "|" Stars:" in cast_info:
        # Director 추출
        director_str = cast_info.split("Director:")[1].split("| Stars:")[0].strip()
        # Stars 추출
        stars_str = cast_info.split("| Stars:")[1].strip()

        # ',' 기준으로 리스트 분리 + 공백 제거
        directors = [d.strip() for d in director_str.split(",") if d.strip()]
        stars = [s.strip() for s in stars_str.split(",") if s.strip()]

    # Title 앞에 있는 숫자 제거
    title = Num_title.split('.',1)[1].strip() if '.' in Num_title else Num_title

    # 결측값 제거 : 빈 문자열이 있으면 해당 행 제외
    if "" in [title, genre, rate] or len(directors) == 0 or len(stars) == 0:
        continue

    # Title에서 Released_Year 추출
    # Title 마지막에 () 형태가 있어야만 슬라이싱
    released_year = None
    if '(' in title and title[-1] == ')':
        released_year = title[title.rfind('(')+1 : title.rfind(')')]
    else:
        # 괄호가 없으면 임의로 None 처리
        released_year = None

    # 타입 변환 후 리스트에 추가
    try:
        processed_data.append([title, genre, float(rate), int(released_year), directors, stars])
    except ValueError:
        # 변환 불가능한 값이 있으면 건너뛰기
        continue

# NumPy 배열로 변환
# dtype=object 사용 : 문자열 + 숫자가 혼합된 배열
data_array = np.array(processed_data, dtype=object)

# 결과 확인
print("NumPy 배열 형태의 데이터 출력: ", data_array)
print("\n데이터 크기(shape) 확인: ", data_array.shape)

# NumPy 배열 바이너리 파일로 저장

```

```
np.save("../03_outputs/results/data_array_extension.npy", data_array)
```

결과

배열 형태의 데이터 출력: [['The Shaws
hank Redemption (1994)' 'Drama' 9.
3 1994
list(['Frank Darabont'])
list(['Tim Robbins', 'Morgan Freema
n', 'Bob Gunton', 'William Sadler'])]
['The Godfather (1972)' 'Crime, Dra
ma' 9.2 1972
list(['Francis Ford Coppola'])
list(['Marlon Brando', 'Al Pacino', 'Ja
mes Caan', 'Diane Keaton'])]
['The Dark Knight (2008)' 'Action, C
rime, Drama' 9.0 2008
list(['Christopher Nolan'])
list(['Christian Bale', 'Heath Ledger',
'Aaron Eckhart', 'Michael Caine'])]
...
['JFK (1991)' 'Drama, History, Thrille
r' 8.0 1991 list(['Oliver Stone'])
list(['Kevin Costner', 'Gary Oldman',
'Jack Lemmon', 'Walter Matthau'])]
['Nights of Cabiria (1957)' 'Drama' 8.
1 1957 list(['Federico Fellini'])
list(['Giulietta Masina', 'François Pér
ier', 'Franca Marzi', 'Dorian Gray'])]
['Throne of Blood (1957)' 'Drama, Hi
story' 8.1 1957
list(['Akira Kurosawa'])
list(['Toshirô Mifune', 'Minoru Chiak
i', 'Isuzu Yamada', 'Takashi Shimur
a'])]]]

데이터 크기(shape) 확인: (887, 6)

```
NumPy 배열 형태의 데이터 출력: [['The Shawshank Redemption (1994)' 'Drama' 9.3 1994  
list(['Frank Darabont'])  
list(['Tim Robbins', 'Morgan Freeman', 'Bob Gunton', 'William Sadler'])]  
['The Godfather (1972)' 'Crime, Drama' 9.2 1972  
list(['Francis Ford Coppola'])  
list(['Marlon Brando', 'Al Pacino', 'James Caan', 'Diane Keaton'])]  
['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008  
list(['Christopher Nolan'])  
list(['Christian Bale', 'Heath Ledger', 'Aaron Eckhart', 'Michael Caine'])]  
...  
['JFK (1991)' 'Drama, History, Thriller' 8.0 1991 list(['Oliver Stone'])  
list(['Kevin Costner', 'Gary Oldman', 'Jack Lemmon', 'Walter Matthau'])]  
['Nights of Cabiria (1957)' 'Drama' 8.1 1957 list(['Federico Fellini'])  
list(['Giulietta Masina', 'François Pérrier', 'Franca Marzi', 'Dorian Gray'])]  
['Throne of Blood (1957)' 'Drama, History' 8.1 1957  
list(['Akira Kurosawa'])  
list(['Toshirô Mifune', 'Minoru Chiaki', 'Isuzu Yamada', 'Takashi Shimura'])]]  
  
데이터 크기(shape) 확인: (887, 6)  
  
종료 코드 0(으)로 완료된 프로세스
```

기존 Step1과 다른 점 / 확장점

항목	기존 Step1	확장 Step1
컬럼 선택	Title, Genre, Rate	Title, Genre, Rate + Director, Star
Released_Year 처리	기존과 동일	주석 명시, int 변환
결측값 제거	기존 컬럼만	확장 컬럼 포함
저장 파일	data_array.npy	data_array_extension.npy
활용	Step2~Step5 분석	감독/배우/조건 필터링 분석 가능

리스트 컴프리헨션에서 if d.strip() / if s.strip() 은 공백만 있는 항목은 무시하고, 실제 이름만 리스트에 넣는다.

확장 Step1: 특정 감독별 평균 평점

🇰🇷 파이참 한글 시각화용 코드 설명

🇮🇹 감독별 평균 평점 상위 10명 시각화 코드 설명

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import numpy as np

# =====
# 파이참 한글 시각화 설정
# =====
# 한글 폰트 경로 설정
font_path = "C:/Windows/Fonts/malgun.ttf" # 맑은 고딕
fontprop = fm.FontProperties(fname=font_path, size=12) # 폰트 속성 객체 생성
```

```

# 2 음수 표시 문제 방지
plt.rcParams['axes.unicode_minus'] = False

# =====
# Step2 Director - 감독별 평균 평점 분석
# =====

# 데이터 로드
data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)

# 감독별 평균 평점 계산
director_ratings = {}
for row in data_array:
    rate = row[2]
    for director in row[4]:
        director_ratings.setdefault(director, []).append(rate)

# 평균 평점 계산
director_avg = {d: round(sum(r)/len(r),2) for d,r in director_ratings.items()}

# 내림차순 정렬
director_sorted = sorted(director_avg.items(), key=lambda x: x[1], reverse=True)

# 상위 10명 출력
print("감독별 평균 평점 상위 10명:")
for d, avg in director_sorted[:10]:
    print(f"{d}: {avg}")

# =====
# 시각화
# =====
top10_directors = director_sorted[:10] # 상위 10명 선택
names = [d[0] for d in top10_directors] # 이름 리스트
scores = [d[1] for d in top10_directors] # 평균 평점 리스트

# 그래프 크기 설정 (가로, 세로) 단위: 인치
plt.figure(figsize=(10,6))

# 수평 막대 그래프 그리기, 리스트 뒤집기 ::-1
plt.barh(names[::-1], scores[::-1], color='skyblue')

# x축 레이블 설정
plt.xlabel("평균 평점", fontproperties=fontprop)

# y축 레이블 설정
plt.ylabel("감독 이름", fontproperties=fontprop)

# 그래프 제목 설정
plt.title("감독별 평균 평점 상위 10명", fontproperties=fontprop)

# 그래프 출력
plt.show()

```

결과

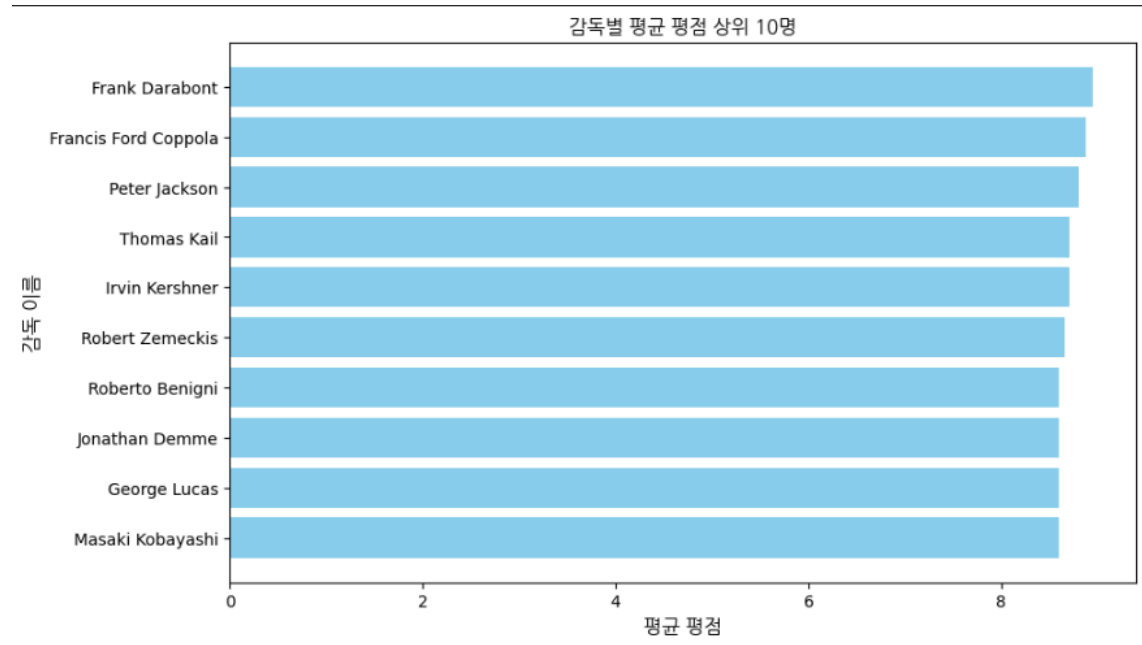
감독별 평균 평점 상위 10명:

Frank Darabont:	8.95
Francis Ford Coppola:	8.87
Peter Jackson:	8.8
Thomas Kail:	8.7
Irvin Kershner:	8.7
Robert Zemeckis:	8.65
Roberto Benigni:	8.6
Jonathan Demme:	8.6
George Lucas:	8.6
Masaki Kobayashi:	8.6

```

감독별 평균 평점 상위 10명:
Frank Darabont: 8.95
Francis Ford Coppola: 8.87
Peter Jackson: 8.8
Thomas Kail: 8.7
Irvin Kershner: 8.7
Robert Zemeckis: 8.65
Roberto Benigni: 8.6
Jonathan Demme: 8.6
George Lucas: 8.6
Masaki Kobayashi: 8.6

```



확장 Step2: 특정 배우별 평균 평점

 배우별 평균 평점 상위 10명 시각화 코드 설명

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import numpy as np

# =====
# 파이썬 한글 시각화 설정
# =====
# 한글 폰트 경로 설정
font_path = "C:/Windows/Fonts/malgun.ttf" # 맑은 고딕
fontprop = fm.FontProperties(fname=font_path, size=12) # 폰트 속성 객체 생성

# 2 음수 표시 문제 방지
plt.rcParams['axes.unicode_minus'] = False

# =====
# Step3 star - 배우별 평균 평점 분석
# =====

# 데이터 로드
data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)

# 배우별 평점 수집
star_ratings = {}
for row in data_array:
    rate = row[2]
    for star in row[5]:
        star_ratings.setdefault(star, []).append(rate)

# 배우별 평균 평점 계산
star_avg = {s: round(sum(r)/len(r), 2) for s,r in star_ratings.items()}

# 평균 평점 기준 내림차순 정렬
star_sorted = sorted(star_avg.items(), key=lambda x: x[1], reverse=True)

# 상위 10명 출력
print("배우별 평균 평점 상위 10명:")
for s, avg in star_sorted[:10]:
    print(f"{s}: {avg}")

# =====
# 시각화
# =====
top10_stars = star_sorted[:10] # 상위 10명 배우 선택
names = [s[0] for s in top10_stars] # 배우 이름만 추출
scores = [s[1] for s in top10_stars] # 평균 평점만 추출

plt.figure(figsize=(10,6))
plt.barh(names[::-1], scores[::-1], color='salmon')
plt.xlabel("평균 평점", fontproperties=fontprop)
```

```
plt.ylabel("배우 이름", fontproperties=fontprop)
plt.title("배우별 평균 평점 상위 10명", fontproperties=fontprop)
plt.show()
```

결과

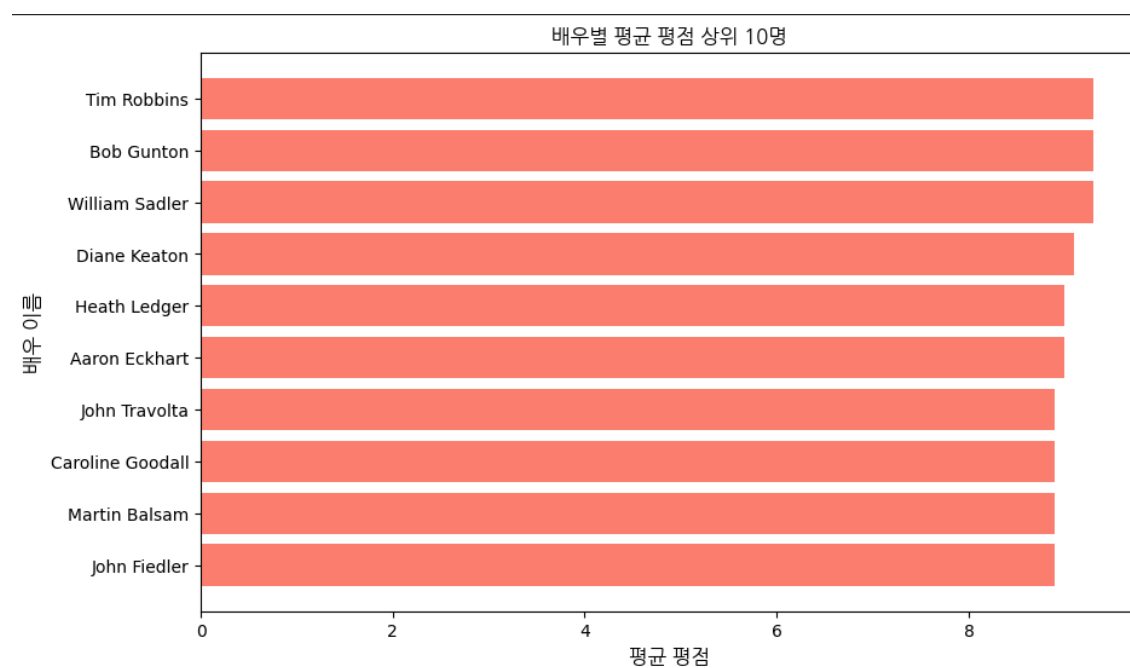
배우별 평균 평점 상위 10명:

Tim Robbins: 9.3
 Bob Gunton: 9.3
 William Sadler: 9.3
 Diane Keaton: 9.1
 Heath Ledger: 9.0
 Aaron Eckhart: 9.0
 John Travolta: 8.9
 Caroline Goodall: 8.9
 Martin Balsam: 8.9
 John Fiedler: 8.9

배우별 평균 평점 상위 10명:

Tim Robbins: 9.3
 Bob Gunton: 9.3
 William Sadler: 9.3
 Diane Keaton: 9.1
 Heath Ledger: 9.0
 Aaron Eckhart: 9.0
 John Travolta: 8.9
 Caroline Goodall: 8.9
 Martin Balsam: 8.9
 John Fiedler: 8.9

종료 코드 0(으)로 완료된 프로세스



확장 Step3: 특정 조건 충족하는 영화만 필터링

✂ 조건 필터링: 평점/제작년도 기준 영화 추출 코드 설명

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import numpy as np

# =====
# 파이참 한글 시각화 설정
# =====
# 한글 폰트 경로 설정
font_path = "C:/Windows/Fonts/malgun.ttf" # 맑은 고딕
fontprop = fm.FontProperties(fname=font_path, size=12) # 폰트 속성 객체 생성

# 2 음수 표시 문제 방지
plt.rcParams['axes.unicode_minus'] = False

# =====
# Step4 filter - 특정 조건 충족하는 영화만 필터링
# =====

# 데이터 로드
data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)

# 조건 필터링
min_rate = 8.0
min_year = 2000

filtered = []
```



```
for row in data_array:
    title, rate, year = row[0], row[2], row[3]
    if rate >= min_rate and year >= min_year:
        filtered.append([title, rate, year])

filtered_array = np.array(filtered, dtype=object)

# 결과 출력
print(f"평점 {min_rate} 이상 & {min_year}년 이후 제작 영화 {len(filtered)}개:")
for movie in filtered_array[:20]: # 상위 20개만 출력
    print(f"{movie[0]}")

# =====
# 시각화
# =====
titles = [f[0] for f in filtered_array[:10]] # 상위 10개 영화 제목
rates = [f[1] for f in filtered_array[:10]] # 평점

plt.figure(figsize=(10,6))
plt.barh(titles[::-1], rates[::-1], color='lightgreen')
plt.xlabel("평점", fontproperties=fontprop)
plt.ylabel("영화 제목", fontproperties=fontprop)
plt.title("평점 8.0 이상 & 2000년 이후 제작 영화 상위 10", fontproperties=fontprop)
plt.show()
```

결과

평점 8.0 이상 & 2000년 이후 제작 영화 496개:
The Dark Knight (2008))
The Lord of the Rings: The Return of the King (2003))
Inception (2010))
The Lord of the Rings: The Fellowship of the Ring (2001))
Hamilton (2020))
The Lord of the Rings: The Two Towers (2002))
Parasite (2019))
Interstellar (2014))
Spirited Away (2001))
Joker (2019))
Whiplash (2014))
The Prestige (2006))
The Departed (2006))
The Pianist (2002))
Gladiator (2000))
Capernaum (2018))
Ayla: The Daughter of War (2017))
Your Name. (2016))
Dangal (2016))
Django Unchained (2012))

종료 코드 0(으)로 완료된 프로세스

```
평점 8.0 이상 & 2000년 이후 제작 영화 496개:
The Dark Knight (2008))
The Lord of the Rings: The Return of the King (2003))
Inception (2010))
The Lord of the Rings: The Fellowship of the Ring (2001))
Hamilton (2020))
The Lord of the Rings: The Two Towers (2002))
Parasite (2019))
Interstellar (2014))
Spirited Away (2001))
Joker (2019))
Whiplash (2014))
The Prestige (2006))
The Departed (2006))
The Pianist (2002))
Gladiator (2000))
Capernaum (2018))
Ayla: The Daughter of War (2017))
Your Name. (2016))
Dangal (2016))
Django Unchained (2012))

종료 코드 0(으)로 완료된 프로세스
```

